

CSE233 Week 4: While and For Loops

This week we will discuss while and for loops. Last week, we discussed if statements, which run a set of statements one time if a given condition is true. Loops are used to run a set of statements multiple times if a condition is true. Unlike if statements, while and for loops do not have an else clause. Once the loop is done, it is done. First, we'll look at the increment and decrement operators, since they are often used with loops.

Pre- and Post-Increment/Decrement Operators

Two operators we did not discuss in Week 2 are the increment and decrement operators, `++` and `--`, respectively. The `++` operator increases a numeric variable by 1, while `--` decreases a numeric variable by 1. The `++` or `--` can go before or after the name of the variable. There cannot be a space between the two plus or minus signs, but there can be a space between the operator and the variable (but just because there can be, doesn't mean there should. Please never let me see `++ x`). Below is an example:

```
// x starts at 0
int x = 0;
// x is increased to 1
++x;
// x is increased to 2
x++;
// x is decreased back to 1
--x;
// x is increased to 2
++x;
// x is decreased to 1
x--;
// x is decreased to 0
--x;
```

The main difference between putting the `++` (or `--`) before or after the variable name is when the variable's value gets read. When using it before, the value is incremented (or decremented) first, then the value of the variable is returned. When using them after the name, the value of the variable gets returned, then it gets incremented (or decremented). Let's look at an example:

```
int x = 0;
// in this case y will be 1
// the value of x is increased first,
// then returned and stored into y
int y = ++x;

int a = 0;
// here, b will be 0
// first, the value of a, 0, is returned and stored in b
// then, the value of a is increased to 1 afterwards
int b = a++;
```

Since using `++` or `--` first does not require storing the original value of the variable being modified, it is typically faster to use that version of the operator. There are some instances where using `++` after is more efficient, but these cases are rare.

While Loops

The basic syntax for the while loop is shown below:

```
while (condition) {
    statement;
}
```

Like the if statement, when the body of the while loops is only a single statement, the curly braces become optional. The condition on the while loop is a Boolean expression, just like the condition for an if statement. There are two main types of loop: definite and indefinite. Definite loops use a counter to determine when to exit. The number of iterations for the loop can be a number that is written into the code, or read from the user. Indefinite loops use a sentinel value to determine when the loop is finished.

Definite While Loops

Below is a basic definite loop:

```
// set the number of times the loop will run
const int num_iterations = 5;
// set a counter variable to start at 0
int i = 0;

// when starting at 0, use less than to make the
// number of times the loop runs equal to num_iterations
while (i < num_iterations) {
    // printing out the current value of i
    std::cout << "i = " << i << std::endl;
    // increment i
    ++i;
}
// print value of i after the loop
std::cout << "After the loop, i = " << i << std::endl;
```

First, we've declared a constant variable to store the number of iterations for the loop. Then, we declare a counter variable, `i`, to count the number of times the loop has executed so far. Typically, in programming, counters start at 0.

Next, we have the while loop. We put the keyword `while`, followed by a set of parenthesis. Inside the parentheses, we have the condition for the while loop, `i < num_iterations`. In this case, the body of the while loop executes so long as the value of `i` remains strictly less than `num_iterations`.

Inside the body of the loop, we have a statement that prints out the current value of `i`. Then, we have the statement `++i`, which increases the value of `i` by one each time the body of the loop runs. This is important; if we didn't increment the value of `i`, its value would remain at 0. 0 is always less than 5, the value of `num_iteration`, meaning that this loop would run infinitely. Infinite loops are undesirable.

Let's analyze the output of this code snippet.

- The first time the while loop is encountered, the value of `i` is 0. 0 is less than 5, so the loop body executes, and "i = 0" is printed. The value of `i` is then increased to 1.
- We then go back to the condition and check it again. `i` is now 1. 1

is still less than 5, so the loop executes again. This time, "i = 1" is printed, and `i`'s value is increased to 2.

- 2 is less than 5, so the loop executes again. "i = 2" is printed and `i` gets incremented to 3.
- 3 is less than 5, so the loop executes again. "i = 3" is printed and `i` gets incremented to 4.
- 4 is less than 5, so the loop executes again. "i = 4" is printed and `i` gets incremented to 5.
- Now, when we check the condition, `i` is 5, and 5 is not less than 5. Therefore, instead of executing the body of the loop, we move to the next line of code after the loop. In this case, the next line is another `std::cout` statement that prints: "After the loop, i = 5".
- The rest of the program will proceed from there.

A definite loop can also have its maximum value read in from the user:

```
// read in the total number of numbers from the user
int total;
std::cout << "Enter how many numbers will be input: ";
std::cin >> total;

// initialize counter and sum
int i = 0;
double sum = 0;

// run the loop while i is less than the total
while (i < total) {
    double num;
    std::cout << "Enter a number: ";
    std::cin >> num;

    // add to sum
    sum += num;
    // increase counter
    i += 1;
}

// compute the average
double average = sum / total;

// print the average
std::cout << "The average is " << average << std::endl;
```

The counter for a definite loop also does not have to increase. You can

also have a loop that starts at its maximum value and counts down. Below is an example that will print the "99 bottles of DRINK on the wall". The user can enter the drink they want printed and how many times to print the verse of the song.

```
// reading in the drink choice
std::string drink;
std::cout << "Enter drink: ";
std::cin >> drink;

// reading in the number of bottles
int bottles;
std::cout << "Enter number of bottles: ";
std::cin >> bottles;

// line that separates the input from the song
std::cout << "\n-----\n";

// while loop that prints the song
// since we are counting down, we want our control variable to be
// greater than the minimum, which is 0 in this case
while (bottles > 0) {
    // print the first set of lyrics in a verse
    std::cout << bottles << " bottles of " << drink << "on the wall\n"
    std::cout << bottles << " bottles of " << drink;
    std::cout << "Take one down, pass it around\n";
    // since the next line of the song has 1 fewer bottle
    // we'll decrement that value of bottles here.
    --bottles;
    std::cout << bottles << " bottles of " << drink << "on the wall\n"
    // print an extra blank line to separate the verses
    std::cout << "\n";
}
```

Indefinite Loops

In the definite loops, we knew how many times the loop would execute based on either a value we coded into the program or read from the user. Then, we could either count up to that value, or count down from it to know when to stop the loop. Sometimes, you don't have a way of knowing how many times a loop will be run. This is especially true when reading user input or reading from a file. In this case, an indefinite loop can be used. Indefinite loops use a sentinel value to determine when to stop running. The sentinel value can be something that you decide, or something entered by the user to denote when the loop should stop. In

the example below, the program continues reading names from the user until the value "quit" is entered:

```
// variable to hold user input
std::string input;

// ask user for initial input
std::cout << "Enter a name (or 'quit' to exit): ";
std::cin >> input;

// we are using the value "quit" to determine
// when to exit the program
while (input != "quit") {
    // echo input back to user
    std::cout << "You entered: " << input << std::endl;

    // continue asking user for input
    std::cout << "Enter a name (or 'quit' to exit): ";
    std::cin >> input;
}
```

For Loops

A for loop is a shorthand syntax for a counter controlled while loop. In a single statement, you can initialize your control variable, condition, and increment/decrement the control. Below is the syntax for a for loop.

```
for (init; condition; incr) {
    statement;
}
```

Like the while and if statements, the curly braces are optional when the body of the for loop has only one statement. Note that the `init` and `condition` have a semicolon after them. There is not a semicolon after the `incr`.

The `init` of the for loop can either be setting the value of an existing variable, or the declaration of a whole new variable. When declaring a new variable, the variable is locally scoped to the for loop, and is not available outside that loop. The `condition` is the comparison used to determine if the loop should run. The `incr` is an expression that increases (or decreases) the control variable. Here are the three definite while loop examples rewritten using for loops.

The first example:

```

// set the number of times the loop will run
const int num_iterations = 5;
// we don't need to set a starting value
// we can do it in the for loop
int i;

// i = 0; sets the initial value of i for the loop
// i < num_iterations; is the condition, just like the while
// ++i increases i by 1 each time the loop runs
for (i = 0; i < num_iterations; ++i) {
    // printing out the current value of i
    std::cout << "i = " << i << std::endl;
    // we no longer increment i in the loop
}
// print value of i after the loop
std::cout << "After the loop, i = " << i << std::endl;

```

The second example:

```

// read in the total number of numbers from the user
int total;
std::cout << "Enter how many numbers will be input: ";
std::cin >> total;

// initialize sum
double sum = 0;

// here, we declare i in the for loop since we don't use it after
for (int i = 0; i < total; ++i) {
    double num;
    std::cout << "Enter a number: ";
    std::cin >> num;

    // add to sum
    sum += num;
}

// compute the average
double average = sum / total;

// print the average
std::cout << "The average is " << average << std::endl;

```

And the third example:

```

// reading in the drink choice
std::string drink;
std::cout << "Enter drink: ";

```

```

std::cin >> drink;

// reading in the number of bottles
int bottles;
std::cout << "Enter number of bottles: ";
std::cin >> bottles;

// line that separates the input from the song
std::cout << "\n-----\n";

// since we are counting down, we want our control variable to be
// greater than the minimum, which is 0 in this case
// we can also do "i -= 1" instead of "--i"
for (int i = bottles; i > 0; i -= 1) {
    // print the first set of lyrics in a verse
    std::cout << i << " bottles of " << drink << "on the wall\n";
    std::cout << i << " bottles of " << drink;
    std::cout << "Take one down, pass it around\n";
    // the next line of the song prints one less bottle
    std::cout << i - 1 << " bottles of " << drink << "on the wall\n";
    // print an extra blank line
    std::cout << "\n";
}

```

Break and Continue Statements

We saw break statements last week to exit switch statements. Break statements can also be used to exit a loop early. Continue statements are used to skip to the next iteration of the loop. Here are a couple examples of break and continue being used:

```

// this loop only prints odd numbers
for (int i = 0; i < 10; ++i) {
    if (i % 2 == 0) {
        // skips even numbers
        continue;
    }
    std::cout << "i = " i << std::endl;
}

// this is a definite loop that breaks early
// if the user enters a negative number
int i = 0;
const int max = 10;
while (i < max) {
    double num;
    std::cout << "Enter a positive number: ";
    std::cin >> num;
    // check if number is less than 0

```



```
    if (num < 0) {
        std::cout << "Negative was entered.\n";
        // exits the loop
        break;
    }
}
std::cout << "End of loop.\n";
```

Do While

C++ also offers the do-while loop. The difference between the while and do-while loop is the do-while checks the condition after each iteration of the loop. The basic syntax is below:

```
do {
    statement;
} while (condition);
```

The do-while loop starts with the keyword `do`. The block containing the statements of the loop body follow `do`. After the block, is the `while` keyword with the condition. There needs to be a semicolon after the condition.

Infinite Loops

An infinite loop is a loop whose condition will never be false. Sometimes, this is done intentionally, and a break statement is used to break the loop. In general, though, infinite loops are the result of programmer mistakes. Here are some common errors that cause infinite loops:

- Using the wrong comparison
- Incrementing a counter when you should decrement (and vice versa)
- Using an impossible sentinel value

Examples

Below are some examples of using while and for loops.

Even Numbers from 0 to 100 (While Loop)

```
#include <iostream>
```

```

int main() {
    // initialize counter variable
    int count = 0;
    // loop while the counter is less than or equal to 100
    while (count <= 100) {
        // print the current count
        std::cout << count << std::endl;
        // increase by 2 to only print even values
        count += 2;
    }
}

```

Even Numbers for 0 to 100 (For Loop)

```

#include <iostream>

int main() {
    // in the for loop, the declaration of the counter
    // the condition, and the increment are in the same statement
    for (int i = 0; i <= 100; i+=2) {
        std::cout << i << std::endl;
    }
}

```

Printing a Rectangle (For Loop)

```

#include <iostream>

int main() {
    // outer loop prints handles printing
    // new lines to start the next row
    for (int i = 0; i < 10; ++i) {
        // inner loop prints each row
        for (int j = 0; j < 10; ++j) {
            std::cout << '#';
        }
        std::cout << '\n';
    }
}

```

The output would look like this:

```

#####
#####
#####
#####

```

```
#####  
#####  
#####  
#####  
#####  
#####
```

Grade Summation (While Loops)

This program asks the user to enter a students name. It stops reading students when the name is "quit". It then asks the user to enter grades and assumes all the grades are out of 100. Grades are read until a negative value is entered. The total and average grade percentage is printed for each student.

```
#include <iostream>  
#include <string>  
  
int main() {  
    // read first name  
    std::string name;  
    std::cout << "Enter first student: ";  
    std::cin >> name;  
  
    // run loop while name is not "quit"  
    while (name != "quit") {  
        // initialize variables for computing grades  
        int count = 0;  
        double grade = 0;  
        double total_grade = 0;  
  
        // read first grade  
        std::cout << "Enter first grade for " << name << ": ";  
        std::cin >> grade;  
  
        // read grades while grade is greater than or equal to 0  
        while (grade >= 0) {  
            // increase count  
            ++count;  
            // increse total_grade by the amount entered  
            total_grade += grade;  
            // read the next grade  
            std::cout << "Enter next grade (-1 to stop): ";  
            std::cin >> grade;  
        }  
  
        // compute the average  
        // since we are assuming each grade is out of 100,
```

```

        // we multiply the count by 100
        double average = total_grade / (count * 100);

        // print out the total
        std::cout << "Total of " << count << " grades for "
            << name << " is " << total_grade << "\n";
        // print out the average
        std::cout << "Average grade for " << name << " is "
            << average * 100 << "%\n";

        // read the next name
        std::cout << "Enter next student (or 'quit'):";
        std::cin >> name;
    }

    std::cout << "Bye!\n";
}

```

Conclusion

Loops are used to repeat a set of instructions multiple times. First, we discussed while loops, which execute their body as long as their condition is true. Definite while loops use a counter variable to determine how many times the loop needs to run. Indefinite loops, on the other hand, compare their input to a sentinel value to determine when to stop running. Any definite while loop can also be written as a for loop, which condenses the counter declaration, condition, and increment step into a single statement. Do while loops can be used in situations where the loop body must always run at least once. I don't cover them in too much detail here since they are not available in some other programming languages (such as Python and Rust), and a normal while loop can be used anywhere a do while is used. We also discussed break and continue statements, and common situations that may cause an infinite loop.